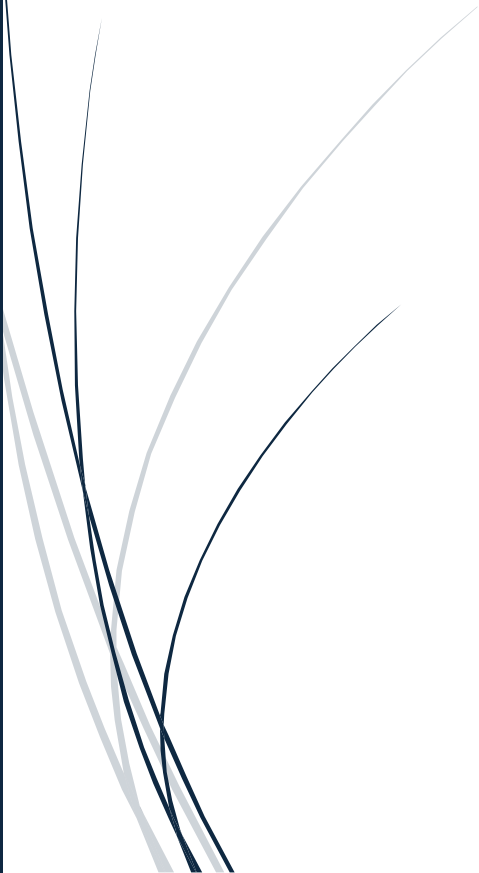




5/7/2024

AI Chatbot with LLM



Seema Kavatkar

CSCI E-104 ADVANCED DEEP LEARNING (HES)

Title: AI Chatbot with LLM

Summary/ Abstract

The project addresses the need for an AI chatbot capable of efficiently retrieving relevant information from large datasets using natural language queries. Leveraging Pinecone's vector search engine and OpenAI's language models, Langchain and Streamlit, we aim to develop a chatbot that provides accurate and contextually relevant responses to user queries. The project focuses on enabling users to search through YouTube encyclopedia using natural language, enhancing accessibility and searchability of video content.

Problem Statement

Despite significant investment in internal support infrastructure, companies still struggle with inefficiencies and delays in resolving employee queries. Traditional support channels like email often lead to bottlenecks and decreased productivity. Complexity in internal processes further hampers timely and accurate support provision. Customized chatbot solutions are increasingly essential to address these challenges, delivering instant assistance, automating tasks, and streamlining workflows. They seamlessly integrate with existing systems, providing valuable support across company processes.

Dataset Source

The dataset, known as "YouTube Wikipedia," is a publicly available encyclopedia on YouTube accessible via the following link: <https://en.wikipedia.org/wiki/YouTube>. For demonstration purposes, we acquired the encyclopedia in PDF format, segmented it into chunks using “RecursiveCharacterTextSplitter”, and subsequently uploaded all these chunks into the Pinecone database using the ‘text-embedding-ada-002’ model from the OpenAPI LLM. The entire document comprises 166 vectors, each representing a chunk detail are elaborated in the report.

Technology/Feature Demonstrated

The key technologies demonstrated include:

- Pinecone DB efficiently store, retrieve, and analyze vast amounts of data.
- OpenAI's LLM extract meaningful insights and perform precise text embeddings.
- Langchain enhances the bot's context memory and response personalization.
- Streamlit for building the chat interface.

The integration of these technologies enables efficient retrieval of semantically relevant information from the dataset, enhancing the chatbot's performance and user experience.

Uses

The AI chatbot facilitates easy access to information within YouTube encyclopedia, allowing users to search for specific content or topics using natural language queries. It enhances accessibility for individuals with hearing impairments, improves search engine optimization (SEO) for video content, and aids in better comprehension of video content.

Benefits

- Efficient retrieval of relevant information from large datasets
- Enhanced accessibility and searchability of video content
- User-friendly interface for seamless interaction
- Accurate and contextually relevant responses to user queries

Drawbacks

- Dependency on the quality and coverage of the dataset
- Potential biases in the data that may affect response accuracy.
- Occasional delays in response time, particularly for complex queries

Challenges

- Optimizing performance and accuracy of the chatbot
- Mitigating bias in the training data and responses
- Minimizing response time delays for improved user experience

Results

Our testing efforts confirm the successful performance of the AI chatbot in providing relevant responses to user queries. The integration of Pinecone and OpenAI's language models contributes to the efficiency and accuracy of information retrieval. The user-friendly chat interface built using Streamlit enhances user engagement and satisfaction.

Working Example

In the working example, users interact with the chatbot via Streamlit, inputting natural language queries related to YouTube encyclopedia.



Author: Seema Kavatkar

1 Table of Contents

1. Introduction	4
2 Installation and Configuration	4
2.1 Setting up Pinecone	4
2.2 Setting up OpenAI	5
2.3 Setting up Streamlit	5
3 Implementation and Execution	6
3.1 Project Setup Instructions	6
3.2 Project File Overview	7
3.3 Architecture	10
3.4 Execution	11
3.4.1 Upload/Upsert data into Pinecone Database	11
3.4.2 Loading chatbot/application via Streamlit	11
3.4.3 User Interface	12
4 Discussion	13
5 Results	14
6 Project Limitations	14
7 Conclusion	15
8 Future Enhancements	16
9 YouTube URLs	16
10 References	17

1. Introduction

In recent years, natural language processing (NLP) and conversational AI technologies have witnessed significant advancements, enabling the development of sophisticated chatbots capable of engaging in human-like conversations. This project explores the integration of Pinecone, a vector search engine, into an AI chatbot built using Streamlit and OpenAI's language models. The integration aims to enhance the chatbot's performance by efficiently retrieving semantically relevant information from a large dataset of dense vector embeddings. This AI Chatbot will allow you to search through YouTube encyclopedia using natural language!

2 Installation and Configuration

2.1 Setting up Pinecone

Pinecone maintains very good documentation on setup and authentication, making it easy for users to get started with their service. Here's an elaboration on the steps provided along with links to Pinecone's documentation:

1. **Obtain a Pinecone API Key:** To begin using Pinecone, you'll need an API key, which you can obtain from the Pinecone dashboard. This key is essential for authenticating your requests to the Pinecone service and accessing its features.
2. **Install the Pinecone Python Client:** Once you have your API key, you can install the Pinecone Python client library using pip, the Python package manager. Simply run the following command in your terminal or command prompt:

Pinecone provides comprehensive documentation to guide users through the setup process and authentication procedures. Here are the relevant links with appropriate captions:

- **Quickstart Guide:** Pinecone's quickstart guide provides a step-by-step walkthrough of setting up Pinecone, creating an index, adding data, and querying for nearest neighbors. This guide is an excellent starting point for getting familiar with Pinecone's capabilities. You can access the quickstart guide here: [Quickstart Guide](#)
- **Authentication Guide:** Pinecone's authentication guide explains how to authenticate your requests to the Pinecone service using your API key. It covers

different authentication methods and best practices for securing your Pinecone account. You can find detailed information on authentication here: [Authentication Guide](#)

By following these resources, users can quickly set up Pinecone, authenticate their requests, and start leveraging Pinecone's powerful features for similarity search and retrieval.

2.2 Setting up OpenAI

Setting up OpenAI is also straightforward, especially with access to its well-documented API and resources. Here's how you can set up OpenAI with similar context to the Pinecone setup:

- **API Key:** Obtain your API key by signing up for an account on the OpenAI website and generating an API key from your account settings.
- **Choose a Client or SDK:** For 'Python' choose the appropriate client library or SDK provided by OpenAI. Choose the **Python 3.11.4** as it best fits project's requirements.
- **Authentication:** Follow the authentication instructions provided in the documentation of your chosen client library or SDK to authenticate your requests using your API key.
- **Explore Documentation:** Dive into OpenAI's documentation to learn more about its APIs, models, and best practices for integrating OpenAI into your applications. Here's the link to OpenAI's API documentation: [OpenAI API Documentation](#)

By following these steps and leveraging OpenAI's resources, you can set up OpenAI and start using its powerful AI models like GPT-3 for various natural language processing tasks.

2.3 Setting up Streamlit

Setting up Streamlit is quite simple, and it's a powerful tool for creating interactive web applications with Python. Here's a link to Streamlit's documentation, which provides comprehensive guides, tutorials, and examples to help you get started: [Streamlit Documentation](#)

By following these steps and referring to Streamlit's documentation, you can quickly set up Streamlit and which is one of the prerequisite of the project to build interactive web applications with Python.

3 Implementation and Execution

3.1 Project Setup Instructions

Prerequisite: Python 3.11.4 (or [download](#))

Assumption: These instructions are tailored for MacOS users.

The setup steps followed for the project are commonly used when working with Python projects, particularly in setting up a virtual environment for managing dependencies and isolating project-specific packages. The steps provided for local setup of the project are as follows:

1. Create a Virtual Environment:

python3 -m venv "ai_chat_bot": This command creates a virtual environment named "ai_chat_bot" using the **venv** module, which comes built-in with Python 3. The virtual environment provides an isolated environment where you can install project-specific dependencies without affecting system-wide packages.

2. Activate the Virtual Environment:

source ai_chat_bot/bin/activate: This command activates the virtual environment named "ai_chat_bot". Activating the virtual environment ensures that any subsequent Python and pip commands use the environment-specific versions of Python and pip, as well as any packages installed within the virtual environment.

3. Check the Virtual Environment:

echo \$VIRTUAL_ENV: This command prints the path to the currently activated virtual environment. It's a useful check to ensure that the virtual environment is activated before proceeding with package installations.

4. Install Required Packages:

pip install -r requirements.txt: This command installs the required Python packages specified in the "requirements.txt" file. This file typically contains a list of dependencies along with their specific versions. Installing dependencies from a requirements file helps maintain consistency across different environments and makes it easier for collaborators to set up the project.

5. Update:

You can simply update the API key in the **.env** file in the private section whenever you need to change it. Here's how you can do it:

- **Locate the .env File:** Navigate to the directory where your project's **.env** file is located.
- **Open the .env File:** Open the **.env** file using a text editor of your choice.
- **Update the API Key:** In the private section of the **.env** file, find the variable corresponding to the API key you want to update (e.g., **OPENAI_API_KEY** or **PINECONE_API_KEY**). Replace the existing API key with the one that acquired in “*Installation and Configuration*” step. Here is an example:

```
# Private Keys Section
# -----
# WARNING: The keys below are masked to protect sensitive information.
# Please replace them with the actual keys before running the project.
# Once the purpose is complete, remove or replace the keys to ensure security.
# Do NOT commit this file to version control systems or share it with unauthorized individuals.
OPENAI_API_KEY= openai_api_key_here
PINECONE_API_KEY= pinecone_api_key_here

# End Private Keys Section
```

Replace **new_openai_api_key_here** and **new_pinecone_api_key_here** with your acquired API keys.

- **Save the .env File:** Save the changes you made to the **.env** file.

By updating the API key in the private section of the **.env** file, you ensure that your Python code continues to use the latest API keys without needing to modify the code itself. This approach helps keep sensitive information separate from your codebase and allows for easy management of API keys.

3.2 Project File Overview

In this section, we provide an overview of the files included in the project. Each file serves a specific purpose and contributes to the overall functionality of the project. Understanding the role of each file is essential for navigating and maintaining the project codebase effectively.

#	File Path	Description
1.	.env	The <code>.env`</code> file is used to store environment variables for the project. These variables typically include sensitive information such as API keys, database credentials, and other configuration

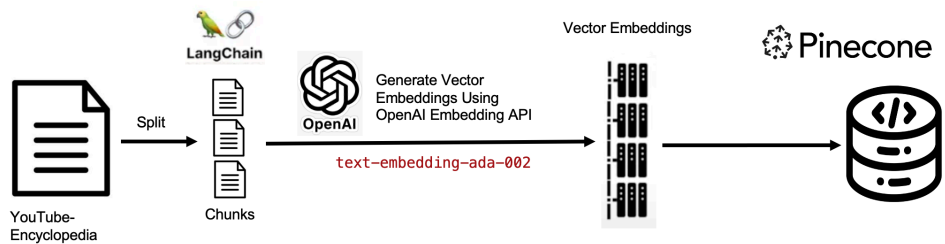
		settings. By storing this information in the <code>.env</code> file, it is kept separate from the codebase and can be easily managed across different environments (e.g., development, testing, production) without exposing sensitive data. The <code>.env</code> file follows a simple key-value pair format, where each line represents a single environment variable in the form of <code>KEY=VALUE</code> .
2.	dataset/	The data repository is a centralized storage hub housing essential project documents which are going to servers as dataset, currently including the "YouTubeEncyclopedia.pdf" file. This document serves as a primary resource for the project's demonstration, containing comprehensive information about YouTube.
3.	Indexing.ipynb	"indexing.py" is a Python script designed to create a Pinecone index and upsert a dataset. It utilizes Pinecone and OpenAI's embeddings, along with PyPDF2 for PDF handling. The script reads PDF documents, chunks them for efficiency, generates embeddings using an embeddings model, and then upserts them into the Pinecone index. To execute the script, simply run it in a Python environment after providing appropriate input parameters such as directory paths and index names.
4.	requirements.txt	The <code>requirements.txt</code> file is commonly used in Python projects to specify the dependencies required for the project to run. Each line in the file typically lists a package name along with its version number or version range. These dependencies include third-party libraries, frameworks, and modules that the project relies on. When setting up the project environment, developers can use the <code>requirements.txt</code> file to install all required dependencies at once using a package manager like pip. This file ensures consistency across different development environments and facilitates collaboration by providing a clear list of dependencies required for the project.
5.	app.py	The <code>app.py</code> file is the main script that defines and runs a web application using the Streamlit framework. Streamlit is a Python library that simplifies the process of creating interactive web

		applications for data science and machine learning projects. In <code>app.py</code>, developers define the layout and functionality of the web application using Streamlit's intuitive Python API. This includes creating interactive widgets such as sliders, buttons, and dropdowns, displaying data visualizations, and processing user inputs. Streamlit automatically updates the web application in real-time as the code is modified, allowing for rapid prototyping and iteration. The <code>app.py</code> file typically serves as the entry point to the Streamlit application, and running this file launches the web application locally, making it accessible through a web browser. Streamlit's ease of use and flexibility make it a popular choice for building data-driven web applications with minimal effort.
6.	<code>utils.py</code>	The <code>utils.py</code> file typically contains utility functions and helper methods that are used throughout the project. These functions often perform common tasks or implement reusable functionality that is needed across multiple modules or components of the project. By centralizing utility functions in a separate file, developers can maintain a clean and organized codebase, avoid code duplication, and promote code reusability. Common types of utility functions found in <code>utils.py</code> files include data preprocessing functions, file I/O operations, data validation functions, and data retrievals. Additionally, <code>utils.py</code> may also include constants, configurations, or custom data structures that are used across the project. Organizing utility functions in a dedicated file helps improve code readability and maintainability, making it easier for developers to collaborate on the project and understand the overall functionality of the codebase.
7.	<code>test.py</code> (optional)	The <code>test.py</code> file typically contains unit tests or test cases that are designed to verify the correctness and robustness of the project's codebase. Unit testing is a software testing technique where individual units or components of a software application are tested in isolation to

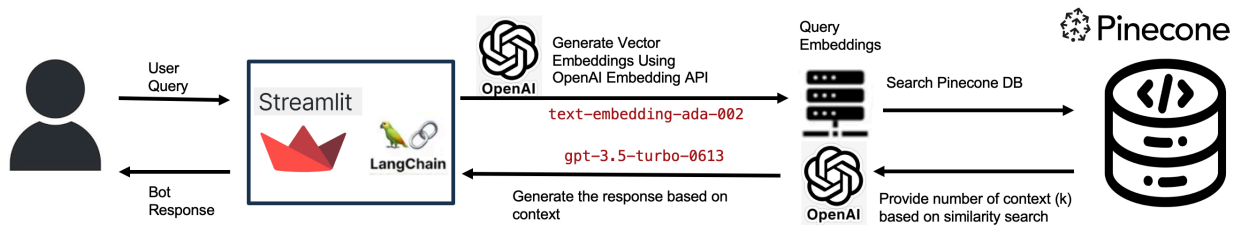
		ensure they function as expected. In <code>test.py</code> , developers define test cases that exercise specific functionalities for pinecone database response, checking for expected behaviors, edge cases, and error conditions. These tests help identify bugs, regressions, and performance issues early in the development process, ensuring the reliability and stability of the software.
--	--	--

3.3 Architecture

Step 1: Create Embedding for the File Text, Then Save Embeddings into Pinecone Vector DB



Step 2: Create Embedding for the Query, Then Retrieve Results from Pinecone Vector DB



* In the demo if no response is provided, Bot is going to explore OpenAI's capabilities

@ Kavatkar, Seema

4

Here is how architecture of this bot look like: This architecture enables users to leverage the combined capabilities of Pinecone's vector database and OpenAI's LLM API to perform semantic search and retrieve relevant information from their own Pinecone DB efficiently.

As a first step, we fetch PDF documents from a repository, convert them into small chunks using the OpenAI embedding model and upsert them in Pinecone database with the index name 'youtube-tech' if it doesn't exist.

Now, when a user posts a query, the script converts the query into a vector using the same embedding model and searches the vector database. When it identifies relevant

documents or passages, the retrieved results are then returned to the user via the LLM API in natural language format, providing answers or insights.

This architecture enables to leverage the combined capabilities of Pinecone, OpenAI's LLM, Langchain and Streamlit to store, perform semantic search and retrieve relevant information efficiently.

3.4 Execution

After successfully completing 'Project Setup Instructions' from this report we are now ready to see the working software.

3.4.1 Upload/Upsert data into Pinecone Database

In Pinecone, data upsert refers to the process of adding or updating vectors (data points) in an index. This ensures that the index reflects the latest information and can provide accurate and relevant results for queries. The upsert operation involves sending new or modified vectors along with their unique identifiers to the index, where they are inserted or updated as needed. This process is crucial for maintaining data integrity and optimizing search performance in real-time applications. Here is a terminal command to set up pinecone database with the demo dataset (youtube-tech)

```
bash
```

```
python indexing.py
```

3.4.2 Loading chatbot/application via Streamlit

The `app.py` file typically serves as the entry point to the Streamlit application, and running this file launches the web application locally, making it accessible through a web browser. The `app.py` file is the main script that defines and runs a web application using the Streamlit framework. Streamlit is a Python library that simplifies the process of creating interactive web applications for data science and machine learning projects. In `app.py`, developers define the layout and functionality of the web application using Streamlit's intuitive Python API. This includes creating interactive widgets such as sliders, buttons, and dropdowns, displaying data visualizations, and processing user inputs. Streamlit automatically updates the web application in real-time as the code is modified, allowing for rapid prototyping and iteration. Streamlit's ease of use and flexibility make it a popular

choice for building data-driven web applications with minimal effort. Here is a terminal command to launch the chatbot:

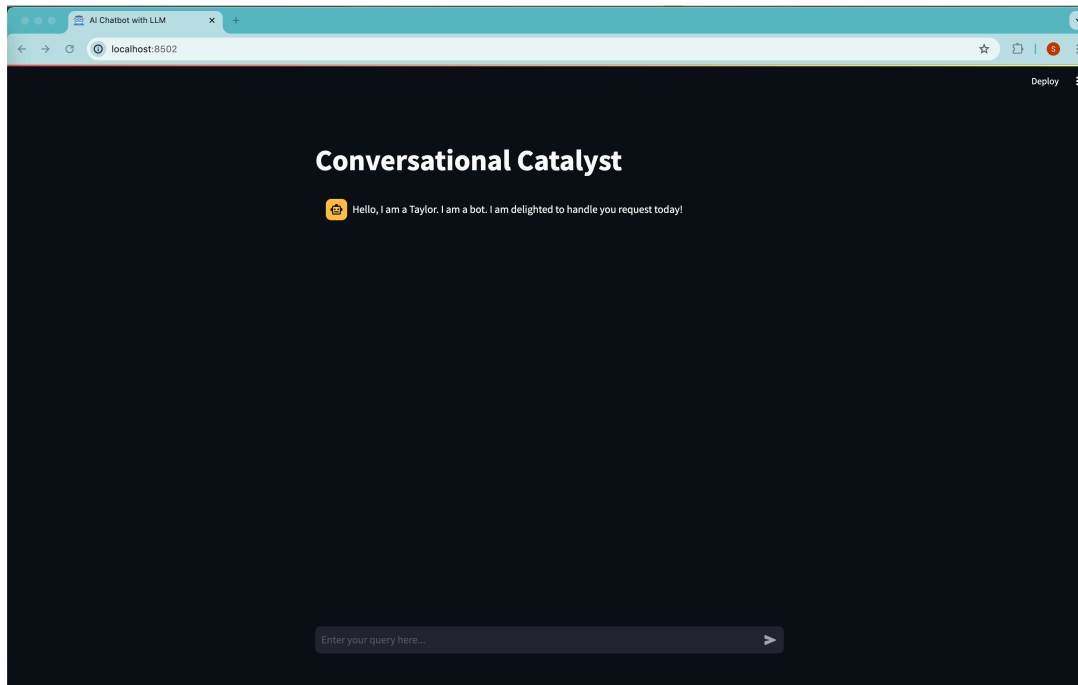
```
bash

streamlit run app.py
```

Note: *It is important to maintain session state for tracking conversation history.*

3.4.3 User Interface

Here is the user Interface of chat bot you will see after loading the chatbot via Streamlit framework.



Sample Q&As:

Here are some sample questions and answers, along with their references in the 'YouTube Encyclopedia' document. When comparing the responses to the encyclopedia, we observe an impressive level of accuracy. Notably, question number 3 showcases the bot's ability to accurately respond to an incomplete question, drawing insights from the previous context. The beauty lies in the seamless integration of technology and human-like understanding, enabling the computer to interpret and respond intelligently, almost as if it's reading the mind.

AI Chatbot with LLM

The screenshot displays a web application titled "Conversational Catalyst" running on a browser at localhost:8502. The chatbot interface on the left shows a series of questions and answers about YouTube, numbered 1 through 4. The questions are: "YouTube is own by whom?", "On which year YouTube was purchased by google?", "Type of business?", and "When is the YouTube founded?". The answers are: "YouTube is owned by Google.", "YouTube was purchased by Google in 2006.", "The type of business is a subsidiary.", and "YouTube was founded on February 14, 2005." To the right of the chatbot is a Wikipedia article titled "YouTube" with a table of contents and a list of references. The table of contents includes sections like "Type of business", "Type of site", "Founded", "Headquarters", "Area served", "Owner", and "Founder(s)". The list of references includes a link to the Wikipedia article and a link to the YouTube website.

Conversational Catalyst

Hello, I am a Taylor. I am a bot. I am delighted to handle you request today!

1. YouTube is own by whom?
YouTube is owned by Google.
2. On which year YouTube was purchased by google?
YouTube was purchased by Google in 2006.
3. Type of business?
The type of business is a subsidiary.
4. When is the YouTube founded?
YouTube was founded on February 14, 2005.

Enter your query here...

WIKIPEDIA
The Free Encyclopedia

YouTube

1. **YouTube** is an American online video sharing platform owned by Google. Accessible worldwide,^[note 1] it was launched on February 14, 2005, by Steve Chen, Chad Hurley, and Jawed Karim, three former employees of PayPal. Headquartered in San Bruno, California, United States, it is the second most visited website in the world, after Google Search. YouTube has more than 2.5 billion monthly users,^[2] who collectively watch more than one billion hours of videos every day.^[7] As of May 2019, videos were being uploaded to the platform at a rate of more than 500 hours of content per minute,^{[8][9]} and as of 2021, there were approximately 14 billion videos in total.^[9]
2. In October 2006, YouTube was purchased by Google for \$1.65 billion (equivalent to \$2.31 billion in 2023).^[10] Google expanded YouTube's business model of generating revenue from advertisements alone, to offering paid content such as movies and exclusive content produced by and for YouTube. It also offers YouTube Premium, a paid subscription option for watching content without ads. YouTube incorporated Google's AdSense program, generating more revenue for both YouTube and approved content creators. In 2022, YouTube's annual advertising revenue increased to \$29.2 billion, more than \$9 billion higher than in 2020.^{[11][12]}

Since its purchase by Google, YouTube has expanded beyond the core website into mobile apps, network television, and the ability to link with other platforms. Video categories on YouTube include music videos

YouTube
Logo used since 2017

Screenshot of the video player on September 13, 2023

Type of business	Subsidiary
Type of site	Online video platform
Founded	February 14, 2005
Headquarters	901 Cherry Avenue San Bruno, California, United States
Area served	Worldwide (excluding blocked countries)
Owner	Google LLC
Founder(s)	Steve Chen Chad Hurley Jawed Karim

4 Discussion

The integration of Pinecone significantly enhances the chatbot's capabilities by enabling fast and accurate retrieval of semantically similar text passages. However, challenges such as optimizing query performance and handling out-of-domain queries remain areas for further improvement. Overall, the project demonstrates the feasibility and effectiveness of combining advanced language models with vector search technology.

In our project, we employed multiple Large Language Models (LLMs) to cater to different aspects of the chatbot's functionality. Building more reliable LLMs tools requires an external "*Knowledge Base*", a place where we can store and use to efficiently retrieve information. We can think of this as the external *long-term memory* of our LLM.

We will need to retrieve information that is semantically related to our queries, to do this we need to use "*dense vector embeddings*". These can be thought of as numerical representations of the *meaning* behind our sentences.

There are many options for creating these dense vectors, like:

- A) Open source [sentence transformers](#)
- B) OpenAI's [ada-002 model](#).

We leverage OpenAI's offering in this project.

By leveraging these diverse LLM models, each tailored to specific tasks within the chatbot system, we ensure the robustness, efficiency, and effectiveness of our AI-powered conversational interface.

5 Results

Our testing efforts have confirmed the successful performance of the AI chatbot in responding to user queries with relevant information retrieved from the dataset. The integration of Pinecone has notably contributed to the efficiency of retrieving dense vector embeddings, consequently enhancing the accuracy and contextual relevance of the bot's responses. This integration ensures that the chatbot can swiftly and accurately provide meaningful information to users based on their queries.

Moreover, the chat interface designed for user interaction has been instrumental in providing a seamless and user-friendly experience. Users can engage in natural conversations with the bot, facilitating smooth interactions and intuitive communication. The interface's design and functionality enable users to comfortably interact with the chatbot, enhancing user engagement and satisfaction.

Overall, the project results demonstrate the successful implementation of an AI chatbot system capable of effectively responding to user queries, leveraging advanced techniques such as dense vector embeddings retrieval through Pinecone integration. The user-friendly chat interface further enhances the overall user experience, making the interaction with the bot intuitive and engaging. These outcomes validate the effectiveness and practicality of the developed AI chatbot system in providing accurate and contextually relevant responses to users.

6 Project Limitations

- Despite the effectiveness of our chatbot, it's essential to acknowledge its limitations. One notable limitation is its dependency on the quality and coverage of the training dataset. While our chatbot performs well within the scope of the provided dataset, its responses may lack accuracy or relevance when presented with queries outside the dataset's domain. Addressing this limitation would require expanding the dataset and continuously updating it to cover a broader range of topics and contexts.
- Another limitation is the potential bias inherent in the training data, which could result in biased or inappropriate responses from the chatbot. As with any AI system,

mitigating bias requires careful curation of the training data and ongoing monitoring of the chatbot's responses to detect and address bias-related issues.

- One limitation observed during testing is the potential for delays in the chatbot's response time, particularly when handling complex queries or performing retrieval operations from external sources such as Pinecone. This slowness of response may arise due to factors such as computational overhead in processing large datasets, latency in accessing external APIs, or limitations in the hardware infrastructure hosting the chatbot system. The occasional delays in response time may still occur, impacting the user experience, particularly in scenarios where timely responses are crucial. Addressing this limitation requires ongoing optimization and fine-tuning of the chatbot's architecture, resource allocation, and external dependencies to minimize response latency and improve overall system responsiveness.

7 Conclusion

In conclusion, our project represents a significant advancement in the field of conversational AI, leveraging state-of-the-art technologies to develop an AI chatbot capable of engaging in human-like conversations and efficiently retrieving relevant information from a vast dataset of dense vector embeddings. By integrating Pinecone for similarity search and OpenAI's language models for natural language understanding and generation, we have created a powerful and versatile chatbot system that offers users an intuitive and engaging experience.

Through rigorous testing and evaluation, we have validated the effectiveness of our chatbot in providing accurate and contextually relevant responses to user queries. The seamless integration of Pinecone ensures swift retrieval of information, enhancing the chatbot's responsiveness and overall performance. Additionally, the user-friendly chat interface built using Streamlit facilitates smooth interactions and fosters user engagement.

However, our project is not without its limitations. Challenges such as dependency on the quality and coverage of the training dataset, potential biases in the data, and occasional delays in response time highlight areas for further improvement and refinement. Addressing these limitations requires ongoing optimization, continuous monitoring, and the implementation of robust mitigation strategies.

Overall, our project demonstrates the feasibility and effectiveness of combining advanced natural language processing techniques with vector search technology to develop intelligent conversational interfaces. Moving forward, we remain committed to enhancing

our chatbot system, addressing its limitations, and exploring new opportunities to advance the capabilities of conversational AI.

8 Future Enhancements

These are **five** future enhancements which are in the pipeline:

1. **Addressing Limitations:** Prioritizing strategies to overcome dataset dependency, biases, and response delays through continuous system refinement.
2. **Feedback Loop:** Implementing a feedback loop to fine-tune chatbot responses based on user input, enhancing satisfaction and engagement.
3. **Expanded Dataset:** Focusing on broader topic coverage to improve the chatbot's ability to handle diverse queries effectively.
4. **Enhanced Features:** Exploring sentiment analysis and entity recognition for better response tailoring and query understanding.
5. **Advancing AI:** These efforts aim to enhance functionality, performance, and user experience, contributing to the evolution of conversational AI.

9 YouTube URLs

Here are the links for YouTube presentation of this project:

- 2 minutes video URL (short): <https://youtu.be/Hag6gtallZ4>
- 15 minutes video URL(long): <https://youtu.be/xRyKR7yUz9I>

10References

- [1] Pinecone Documentation: <https://docs.pinecone.io/>
 - [2] Streamlit Documentation: <https://docs.streamlit.io/>
 - [3] OpenAI Documentation: <https://beta.openai.com/docs/>
 - [4] YouTube: <https://www.youtube.com/watch?v=zKGeRWjJlTU>
 - [5] Blog: <https://blog.futuresmart.ai/building-an-interactive-chatbot-with-langchain-chatgpt-pinecone-and-streamlit>
-

This report provides a detailed overview of the development process, including installation and configuration steps, implementation details, results, and future directions. The integration of Pinecone, OpenAI, Langchain and Streamlit enhances the chatbot's capabilities, paving the way for more advanced conversational AI systems.